INT161 · Basic Backend Development

Error Handling

Why an unhandled error kills the server, how try and catch work, custom errors, and one Express middleware that answers with the right status code.

What is an Exception | Error Types | Error Propagation | try and catch | Custom Errors | Error-handling Middleware | Status Codes | Assignment | Glossary

01

What is an Exception

An **exception**, or error, is an event that breaks the normal flow of the program. Say the program should run line 1 to 50, and the data makes line 25 fail. The rest cannot run.

- The error is thrown as an object that holds the details. Only Error objects can be thrown.
- If nobody catches it, **Node** stops the program and exits with code 1. A **back end** that stops is an app that is down: the **front end** can no longer call it.
- Errors can happen in every layer, even when your SQL is correct. Example: the database says a column must be unique, and you insert a duplicate.

Why handle them:

- The app does not crash, so it stays up.
- The user gets a friendly message, such as "that value is not valid".
- **Sensitive** data does not leak. An unhandled error sends a **stack trace** that can show table names, or worse.
- It is easier to **debug** and to monitor.

02

Error Types

Programming error

A typo or bad syntax. Rare now, because the IDE underlines it before you run.

Runtime error

Valid code that fails while running. x was a number, five lines later it is text, then you multiply it. JavaScript allows this, so you must catch it.

Logical error

The code runs but the answer is wrong. An exception cannot help. You debug: step through and read the values.

Operational error

The code is fine, the world is not: a missing file, a database that will not connect. Some you expect, some you do not. Catch both.

- Programming and **logical errors** are bugs you fix. **Runtime** and **operational errors** are the ones you catch.
- If you hand a bug to an AI to fix, you still do not know what was wrong, so you learn nothing.

03

Error Propagation

Layers mean one function calls another. main calls a, a calls b. If b **throws** and nobody catches, the error is passed up to a, then main, then the runtime, which stops. You can catch at any level.

test-exception-handling.js

```
import fs from 'node:fs';
function main(value = 0) {
  console.log(a(value));
}
function a(value) {
  return b(value);
}
function b(value) {
  if (value > 0) return ++value;
  return fs.readFileSync('test.txt'); // the file does not exist: throws
}
```

Nowhere | In b() | In a() | In main()

```
main()                                  error passes up
  a()                                   error passes up
    b()                                 error starts here
runtime : stops the program
```

Program

Dies. Exit code 1. Nothing after the error runs.

For a back end

The server stops listening. Every client is cut off.

You can catch at any level. Catching once, high up, means one place to fix.

- In your project, the same chain runs **router**, **service**, **repository**. You may catch in each one, or once at the top in app.js. Catching in one place is easier to maintain.

> ⚠ **Stack trace**
>
> The trace of an uncaught error lists the file and line, and who called whom. Useful for you, but never send it to the **client**.

04

try and catch

Put the risky code in try. If it throws, execution jumps to catch, and the variable you named receives the error object. If nothing throws, catch is skipped.

try and catch

```
try {
  main(1);
  console.log('After main success');
  main(); // throws, so the next line is skipped
  console.log('After main error');
} catch (e) {
  console.log('Message:', e.message);
  console.log('Code:', e.code);
}
console.log('Program ended normally');
```

- main(1) works, then main() throws, so "After main error" never prints. The catch block runs, then the program ends normally with code 0.
- The standard Error object has message (readable text), name (the type) and stack (the trace). Node adds code (such as ENOENT) and errno.
- status is not part of JavaScript. You add it yourself, because it belongs to **HTTP**.
- In a **REST API**, once you catch, you must build the answer: **JSON** with the right **status code**.

05

Custom Errors

You can build an error yourself with new Error and a message. To give every error the same shape, write your own class that extends Error and adds a status code.

AppError

```
class AppError extends Error {
  errors = { 400: 'BAD_REQUEST', 401: 'UNAUTHORIZED', 403: 'FORBIDDEN',
             404: 'NOT_FOUND', 409: 'CONFLICT', 500: 'INTERNAL_SERVER_ERROR' };
  constructor(message, statusCode) {
    super(message); // the message is stored by the Error parent
    this.statusCode = statusCode;
    this.error = this.errors[statusCode];
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}
throw new AppError('Country with id 999 was not found', 404);
```

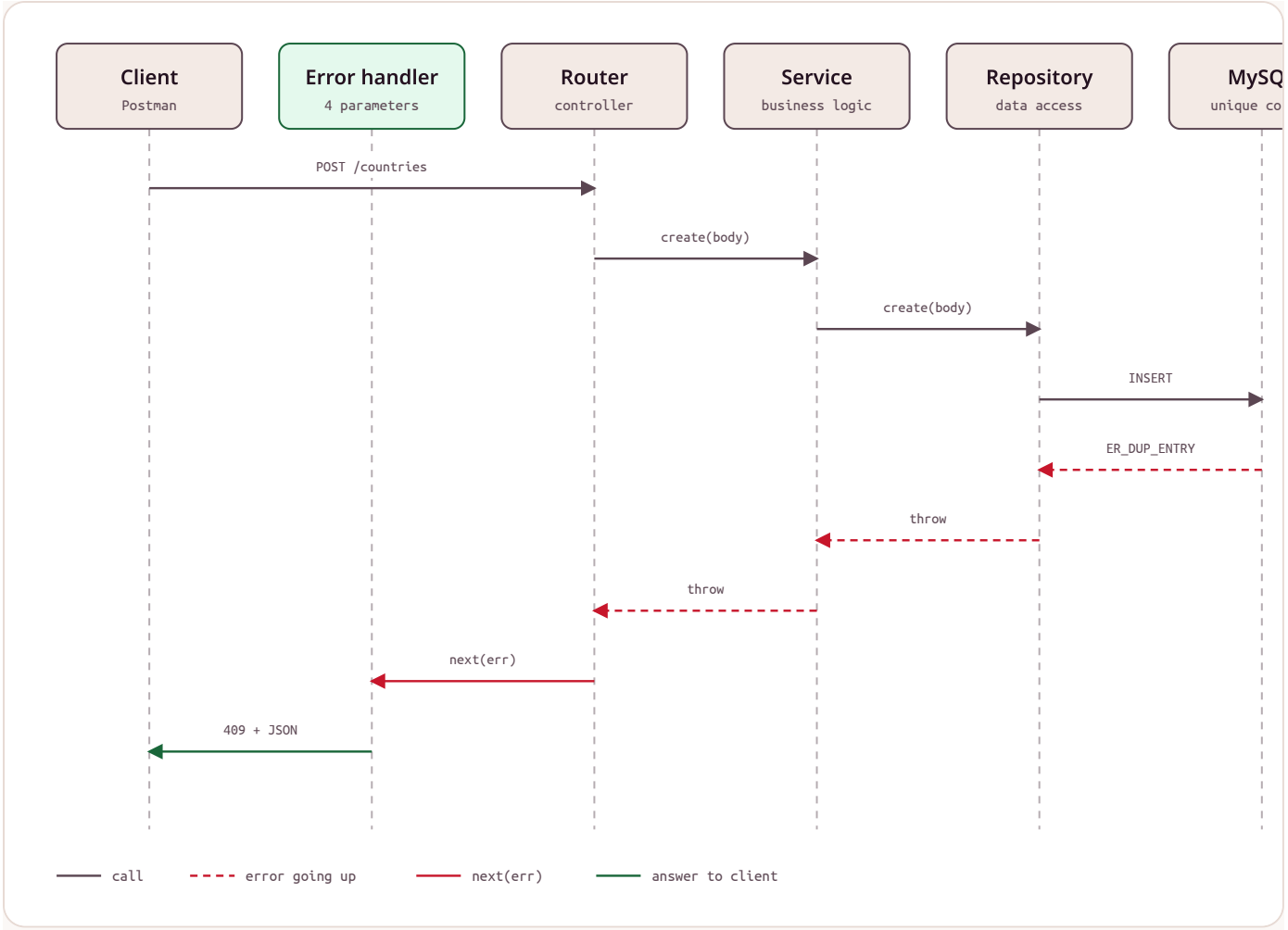
- super(message) hands the text to Error, which stores it in message. The parent owns that field.
- The errors table turns 404 into NOT_FOUND. Pick the 404 and you get the right word.
- The shape of your errors is a design decision between front end and back end. Agree on the fields early. Many Node APIs use a success flag plus data.

06

Error-handling Middleware

Express has a special mechanism. You write one **error-handling middleware**, and every error from every layer is sent to it for you. You do not write **try and catch** in each place.

- A normal **middleware** has (req, res, next). An error-handling one has four: (err, req, res, next). Express runs it only when an error is passed along.



error-handler middleware

```
// error-handling middleware: four parameters
app.use((err, req, res, next) => {
  const status = err.code === 'ER_DUP_ENTRY' ? 409 : err.statusCode || err.status || 500;
  res.status(status).json({
    statusCode: status,
    message: err.message,
    path: req.originalUrl,
    timestamp: new Date().toISOString(),
  });
});
```

- The lecturer added one step: a duplicate-entry error from the database (code ER_DUP_ENTRY) is answered with 409 Conflict. Any other error with a status keeps it. No status at all means 500.

- The answer carries a status code, a readable message, the **URL** that failed (req.originalUrl) and a **timestamp**.
- With a small project, write it in app.js. When it grows, move it to its own file and import it, like a router.

app.js

```
const app = express();
app.use(express.json());
app.use('/countries', countryRouter);
// ... all other routers
app.use(errorHandler); // last, after every router
```

Order matters. Try the three setups.

Handler last | No handler | Handler first

`app.use: express.json()` → `app.use: routers` → `app.use: error handler`

The client gets

JSON, with the right status code

Body

```
{ "statusCode": 409, "message": "Dupli
```

Any error from the **routers** falls through to the handler.

To see it yourself

In the demo project the lecturer made the country column unique in the database, then posted Thailand twice. Without the handler: a 500 and an HTML stack trace. With it: a JSON 409 whose **body** has the status, message, URL and time.

07

Status Codes

The front end learns what happened from the status code, so you must pick the right one. Choose by who made the mistake.

`400 Bad Request`

The client sent something invalid: a text where a number belongs, or a missing body field.

`401 Unauthorized`

Not logged in, or no token.

`403 Forbidden`

Logged in, but this **endpoint** is not for you, such as admin only.

`404 Not Found`

The **resource** does not exist. The one you meet most.

`409 Conflict`

The data clashes with what exists, such as a country that is already there.

`412 Precondition Failed`

A condition in the **request headers** was not met, such as a required header that is missing.

`500 Internal Server Error`

Something broke on our side. Usually avoid sending it on purpose.

`503 Service Unavailable`

The service is not available right now.

- The front end turns the code into a message for the user. These codes also feed the integrated project later.

08

Assignment

The assignment is a markdown file in the week 6 material. Open it in your IDE in preview mode and read it fully. It asks you to start a new project from your template and point it at the classicmodels database.

- Task 1: manage the offices table. When a row is not found, answer in the error format given in the spec.
- Task 2: create product lines, and handle a duplicate entry.
- At least task 1 was due in class. The rest has a later deadline, set on the course homework page.
- Do as the spec says, even if its error format differs from the lecturer's demo. Many Node APIs send a status and a success or error flag.

> **Reuse**
> The assignment hands you a ready-made error class. You can drop it into your middleware. Field names may differ, for example error code versus code, so make the repository and the middleware agree.

09

Glossary

API
: Application Programming Interface: the doors a program offers to other programs.

back end
: The part on the server: it handles data, rules and the database, and answers the front end with JSON.

body
: The data a request carries after the headers. For an API it is usually JSON.

client
: The side that asks. Usually a browser, but it can be any program or device.

debug
: Find and fix a bug, usually by stepping through the code and watching the values.

endpoint
: One method plus path an API answers, such as GET /api/subjects.

error-handling middleware
: A middleware with four parameters (err, req, res, next). Express calls it only when an error is passed along.

exception
: An event that breaks the normal flow of a program. It is thrown as an object holding the details.

Express
: A small web framework for Node.js. It adds routing and middleware on top of the built-in http module.

front end
: The part users see and touch. It runs in the browser or on the phone.

headers
: Extra information about a message, sent as name: value lines, such as Content-Type.

HTTP
: HyperText Transfer Protocol: the plain-text language a browser and a web server use to talk. One side sends a request, the other sends a response.

JSON
: JavaScript Object Notation: a plain text format for data that both humans and programs can read.

logical errors
: The code runs but gives the wrong answer. try and catch cannot help; you must debug.

middleware
: A function (req, res, next) that runs between the request and the route handler. It can change the request, answer it, or pass it on.

Node
: A runtime that runs JavaScript outside the browser, so the same language can write the back end.

operational errors
: The code is fine but the world is not: a missing file, a database that is down.

repository

The layer that reads and writes the database. No business rules, no HTTP.

request

What the client sends: method, URL, headers and sometimes a body.

resource

A thing the API gives access to, such as a product or a customer. In this course it maps to a database table.

REST

A style for web APIs: resources named by URLs, actions chosen by HTTP method.

router

Matching an HTTP method and a URL path to the function that handles it.

routers

Matching an HTTP method and a URL path to the function that handles it.

Runtime

The environment that runs your code. Node.js is a runtime for JavaScript, not a language.

Sensitive

Data that must not leak, such as passwords, card numbers or table names. A stack trace can expose it.

service

The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

stack trace

The list of function calls, with file and line, that led to an error. The top line is where to look first.

status code

The number that tells the client how it went: 200 OK, 201 Created, 404 Not Found, 500 Server Error.

throws

Raise an error. Only Error objects can be thrown.

timestamp

A date and time value. A column like last_update can fill itself with the current time.

try and catch

try runs code that may fail. If it throws, execution jumps to catch, which receives the error.

URL

The address of a resource: protocol, host, port and path, such as http://localhost:3000/user.